Herald

{width=96px height=96px}

Wave 2 — Flavor Scaffolds Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development per Universal Constitution §11.4.70 (subagent-driven default). Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Land 6 new flavor binaries (sherald, cherald, bherald, rherald, iherald, scherald) + a shared `commons/cli/` scaffold + refactor pherald to consume it. e2e_bluff_hunt grows 18 → 33 invariants.

Architecture: Shared `commons/cli/` package provides `NewRootCmd / VersionCmd / ServeCmd / StubCmd` + Healthz/Readyz/Metrics handlers. Each flavor's `cmd/<flavor>herald/main.go` is ~30 LOC: declare `commons.Branding{Flavor, Prefix, DisplayName, DefaultPort, Mission}`, call `cli.NewRootCmd(branding)`, register §43 stubs from `internal/stubs/`, optionally call `cli.ServeCmd(opts)` for serving flavors. pherald refactor unifies its existing surface (version + serve + 501 /v1/events) onto `commons/cli/`; E1-E13 stay green as the §107 regression check.

Tech Stack: Go 1.25+; `github.com/spf13/cobra` (CLI); `github.com/gin-gonic/gin` (HTTP); `github.com/sirupsen/logrus` (logger). All already in pherald's go.mod — moves into `commons/go.mod` for the refactor.

Design ref: docs/superpowers/specs/2026-05-21-wave2-flavor-scaffolds-design.md (commit `6008577`).

Catalogue-Check (Universal §11.4.74): Before writing `commons/cli/`, the implementer MUST search `vasic-digital/` + `HelixDevelopment/` orgs for an existing Cobra+Gin scaffold module. Likely disposition: `no-match → vendor as Herald-internal package`. Document in `docs/catalogue-checks/HRD-092-commons-cli.md` (NEW directory).

File Structure

CREATE

Path	Responsibility
commons/cli/root.go	NewRootCmd(branding, opts...) — Cobra root with Use/Short/Long/Version driven by Branding
commons/cli/version.go	VersionCmd(branding) — version --json subcommand; canonical JSON shape
commons/cli/stubs.go	StubCmd(name, hrd, description) — Cobra subcommand returning 501-style error with HRD pointer
commons/cli/serve.go	ServeCmd(opts) — Cobra subcommand binding Gin engine + healthz/readyz/metrics + custom routes + SIGTERM graceful shutdown
commons/cli/routes.go	Route struct + HealthzHandler / ReadyzHandler / MetricsHandler Gin handlers
commons/cli/cli_test.go	unit tests (table-driven) for all four constructors + handlers
sherald/go.mod + sherald/cmd/sherald/main.go + sherald/internal/stubs/stubs.go + sherald/internal/http/routes.go	sherald flavor
cherald/go.mod + cherald/cmd/cherald/main.go + cherald/internal/stubs/stubs.go + cherald/internal/http/routes.go	cherald flavor
bherald/go.mod + bherald/cmd/bherald/main.go + bherald/internal/stubs/stubs.go	bherald flavor (no serve)
rherald/go.mod + rherald/cmd/rherald/main.go + rherald/internal/stubs/stubs.go	rherald flavor (no serve)
iherald/go.mod + iherald/cmd/iherald/main.go + iherald/internal/stubs/stubs.go + iherald/internal/http/routes.go	iherald flavor
scherald/go.mod + scherald/cmd/scherald/main.go + scherald/internal/stubs/stubs.go	scherald flavor (no serve)
tests/test_wave2_mutation_meta.sh	Paired §1.1 mutation tests for the new e2e invariants
docs/catalogue-checks/HRD-092-commons-cli.md	Catalogue-check evidence for the new commons/cli/ package

MODIFY

Path	Change
`commons/go.mod`	Add `cobra`, `gin`, `logrus` requires + the existing-replace-directive pattern (currently pherald owns these; the move requires shifting deps to commons)
`pherald/cmd/pherald/main.go`	Refactor to consume `cli.NewRootCmd(branding)` instead of constructing Cobra manually
`pherald/cmd/pherald/version.go`	Delete or refactor to thin wrapper that calls `cli.VersionCmd(branding)`
`pherald/cmd/pherald/stubs.go`	Refactor — replace per-stub functions with `cli.StubCmd(...)` registrations
`pherald/cmd/pherald/serve.go` (or wherever the existing serve lives)	Refactor to consume `cli.ServeCmd(opts)` with pherald-specific routes
`pherald/internal/http/routes.go`	Keep flavor-specific routes (e.g. /v1/events) — they're just `cli.Route` slices now
`pherald/go.mod`	Drop now-unused direct deps (cobra/gin/logrus); they come from `commons/cli/` transitively
`go.work`	Add 6 new flavor module directories
`scripts/e2e_bluff_hunt.sh`	Add E19-E33 invariants; bump "Eighteen invariants" → "Thirty-three invariants"
`docs/specs/mvp/specification.V3.md`	§18 Flavors + §41 REST surface + §44 Foundation + §3.5 Branding + §11.4.74 catalogue-checks (Wave 2 captured)
`docs/Issues.md`	Open HRD-092..HRD-097 (one per new flavor; tracks "scaffold complete; awaiting live wiring") + HRD-098 (sherald /v1/safety_state)
`docs/Status.md`	r8 → r9 with Wave 2 completion + e2e count 18 → 33

Task 1: `commons/cli/` — Branding helpers + StubCmd + tests

Why first: StubCmd has no dependencies on other cli/ pieces; isolating it lets us TDD the core 501-pattern before wiring it into anything else.

Files:

- Modify: `commons/types.go` — verify `Branding` struct has Flavor/Prefix/DisplayName/DefaultPort/Mission fields; add if missing
- Create: `commons/cli/stubs.go`
- Create: `commons/cli/cli_test.go`

☐ **Step 1: Inspect existing Branding struct**

```
cd /Users/milosvasic/Projects/Herald
grep -nA 15 "^type Branding" commons/types.go
```

If `DefaultPort` or `Mission` field is missing, add them in this step. The struct already has Flavor + Prefix + DisplayName from HRD-009. Note the exact field shape; the design assumed these exist.

- [ ] **Step 2: Create `commons/cli/cli_test.go` with the StubCmd test**

```
package cli

import (
    "bytes"
    "strings"
    "testing"
)

func TestStubCmd_ReturnsErrorWithHRDPointer(t *testing.T) {
    cmd := StubCmd("destructive-guard", "HRD-033", "wrap rm +
        git-reset with prerequisite checks")
    var stderr bytes.Buffer
    cmd.SetErr(&stderr)
    cmd.SetOut(&stderr)
    err := cmd.Execute()
    if err == nil {
        t.Fatal("expected non-nil error from stub")
    }
    msg := err.Error()
    if !strings.Contains(msg, "HRD-033") {
        t.Errorf("error should contain HRD reference, got: %q",
        msg)
    }
    if !strings.Contains(msg, "destructive-guard") {
        t.Errorf("error should contain command name, got: %q",
        msg)
    }
    if !strings.Contains(strings.ToLower(msg), "not yet
        implemented") {
        t.Errorf("error should explain non-implementation, got:
        %q", msg)
    }
}
```

- [ ] **Step 3: Run test, verify FAIL ("undefined: StubCmd")**

```
go test ./commons/cli/ -count=1 2>&1 | tail -5
```

Expected: compile error.

- [ ] **Step 4: Implement `commons/cli/stubs.go`**

```
// Package cli is the shared CLI scaffold for every Herald flavor
        binary.
// Per Universal §11.4.74 catalogue-check: vendored as Herald-
        internal
// (no-match against vasic-digital + HelixDevelopment).
package cli

import (
    "fmt"

    "github.com/spf13/cobra"
)

// StubCmd returns a Cobra subcommand that always fails with a
        501-style
// error citing the HRD that will implement it. Used by every
        flavor's
// internal/stubs/ to register §43 commands not yet implemented.
//
// Per Herald §107: the error message MUST contain the HRD
        pointer + the
// command name + the literal "not yet implemented" — these are
        the three
// substrings the e2e_bluff_hunt E31 invariant asserts on.
func StubCmd(name, hrd, description string) *cobra.Command {
    return &cobra.Command{
        Use:   name,
        Short: description + " (not yet implemented — " + hrd +
        ")",
        RunE: func(cmd *cobra.Command, args []string) error {
            return fmt.Errorf("%s: not yet implemented — see %s
        for status", name, hrd)
        },
    }
}
```

- [ ] **Step 5: Run test, verify PASS**

```
go test ./commons/cli/ -count=1 -v 2>&1 | tail -10
```

- [ ] **Step 6: Commit**

```
git add commons/cli/stubs.go commons/cli/cli_test.go commons/
        types.go commons/go.mod
git commit -m "Wave 2 step 1: commons/cli/StubCmd — honest 501-
        style command stubs
```

```
First piece of the Wave 2 shared scaffold. StubCmd returns a
        Cobra
subcommand that always fails with the HRD pointer + command name
        +
'not yet implemented' literal — the three substrings
        e2e_bluff_hunt
E31 will assert on per §107.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 2: `commons/cli/` — NewRootCmd + VersionCmd

Files:

- Create: `commons/cli/root.go`
- Create: `commons/cli/version.go`
- Modify: `commons/cli/cli_test.go` (append tests)

☐ **Step 1: Append failing tests**

```go
func TestNewRootCmd_BindsBranding(t *testing.T) {
    br := commons.Branding{
        Flavor:      "sherald",
        Prefix:      "s",
        DisplayName: "System Herald",
        DefaultPort: 24793,
        Mission:     "Host safety + destructive-op intercept",
    }
    cmd := NewRootCmd(br)
    if cmd.Use != "sherald" {
        t.Errorf("Use = %q, want %q", cmd.Use, "sherald")
    }
    if !strings.Contains(cmd.Short, "System Herald") {
        t.Errorf("Short should contain DisplayName, got %q",
        cmd.Short)
    }
}

func TestVersionCmd_JSONOutputShape(t *testing.T) {
    br := commons.Branding{Flavor: "sherald", DisplayName:
        "System Herald"}
    cmd := VersionCmd(br)
    cmd.SetArgs([]string{"--json"})
    var stdout bytes.Buffer
    cmd.SetOut(&stdout)
    if err := cmd.Execute(); err != nil {
        t.Fatalf("Execute: %v", err)
```

```
    }
    var got map[string]any
    if err := json.Unmarshal(stdout.Bytes(), &got); err != nil {
        t.Fatalf("JSON unmarshal failed: %v; raw=%q", err,
        stdout.String())
    }
    for _, key := range []string{"binary", "flavor", "version",
        "go_version", "os", "arch"} {
        v, ok := got[key]
        if !ok {
            t.Errorf("missing key %q in version JSON", key)
        }
        if s, ok := v.(string); ok && s == "" {
            t.Errorf("key %q is empty string — §107 bluff
        guard", key)
        }
    }
    if got["flavor"] != "sherald" {
        t.Errorf("flavor field = %v, want %q", got["flavor"],
        "sherald")
    }
}
```

Add the import: `"encoding/json"` + `"github.com/vasic-digital/herald/commons"`.

- [ ] **Step 2: Run, verify FAIL**

```
go test ./commons/cli/ -count=1 2>&1 | tail -5
```

- [ ] **Step 3: Implement `commons/cli/root.go`**

```
package cli

import (
    "fmt"

    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons"
)

// RootOpt is a functional option for NewRootCmd.
type RootOpt func(*cobra.Command)

// NewRootCmd returns the top-level Cobra command for a flavor.
        Branding
// drives Use / Short / Long / Version output.
```

```
func NewRootCmd(br commons.Branding, opts ...RootOpt)
        *cobra.Command {
    cmd := &cobra.Command{
        Use:   br.Flavor,
        Short: fmt.Sprintf("%s — %s", br.DisplayName,
        br.Mission),
        Long: fmt.Sprintf(
            "%s (%s).\n\n%s\n\nFlavor prefix: %s — see Helix
        Universal Constitution §8.2.",
            br.DisplayName, br.Flavor, br.Mission, br.Prefix,
        ),
        SilenceUsage:  true,
        SilenceErrors: true,
    }
    for _, opt := range opts {
        opt(cmd)
    }
    return cmd
}
```

- [ ] **Step 4: Implement** `commons/cli/version.go`

```
package cli

import (
    "encoding/json"
    "fmt"
    "runtime"

    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons"
)

// Build-info variables. Set via -ldflags at build time;
        otherwise default
// to "dev" / unknown markers so the binary is still operational
        from a
// raw `go run` invocation. §107: an empty version is a bluff —
        version_test
// asserts non-empty.
var (
    BuildVersion = "dev"
    BuildCommit  = "unknown"
    BuildTime    = "unknown"
)
```

```
// VersionCmd is the `<flavor>herald version` subcommand. Prints
        human-
// readable build info by default; `--json` returns the canonical
        JSON
// shape used by e2e_bluff_hunt E2/E19-E24.
func VersionCmd(br commons.Branding) *cobra.Command {
    var asJSON bool
    cmd := &cobra.Command{
        Use:   "version",
        Short: "Print " + br.DisplayName + " version + build
        info",
        RunE: func(cmd *cobra.Command, args []string) error {
            info := map[string]string{
                "binary":     br.Flavor,
                "flavor":     br.Flavor,
                "version":    BuildVersion,
                "commit":     BuildCommit,
                "build_time": BuildTime,
                "go_version": runtime.Version(),
                "os":         runtime.GOOS,
                "arch":       runtime.GOARCH,
            }
            if asJSON {
                return
        json.NewEncoder(cmd.OutOrStdout()).Encode(info)
            }
            out := cmd.OutOrStdout()
            fmt.Fprintf(out, "%s %s\n", br.DisplayName,
        BuildVersion)
            fmt.Fprintf(out, "  flavor:     %s\n", br.Flavor)
            fmt.Fprintf(out, "  commit:     %s\n", BuildCommit)
            fmt.Fprintf(out, "  built:      %s\n", BuildTime)
            fmt.Fprintf(out, "  go_version: %s\n",
        runtime.Version())
            fmt.Fprintf(out, "  os/arch:    %s/%s\n",
        runtime.GOOS, runtime.GOARCH)
            return nil
        },
    }
    cmd.Flags().BoolVar(&asJSON, "json", false, "emit JSON
        instead of human-readable text")
    return cmd
}
```

☐ **Step 5: Update go.mod**

```
cd commons
go get github.com/spf13/cobra@latest
```

```
go mod tidy
cd ..
```

- [ ] **Step 6: Run tests, verify PASS**

```
go test ./commons/cli/ -count=1 -v 2>&1 | tail -15
```

- [ ] **Step 7: Commit**

```
git add commons/cli/root.go commons/cli/version.go commons/cli/
        cli_test.go commons/go.mod commons/go.sum
git commit -m "Wave 2 step 2: commons/cli/NewRootCmd + VersionCmd

NewRootCmd reads Use/Short/Long from commons.Branding. VersionCmd
emits canonical JSON when --json passed; required fields (binary,
flavor, version, go_version, os, arch) are populated via runtime
package + ldflags-overridable BuildVersion/BuildCommit/BuildTime.

§107: tests assert non-empty version field — version='' explicitly
forbidden.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 3: `commons/cli/` — Routes + Healthz/Readyz/Metrics handlers

Files:

- Create: `commons/cli/routes.go`
- Modify: `commons/cli/cli_test.go` (append tests)

- [ ] **Step 1: Append failing tests**

```
func TestHealthzHandler_Returns200WithBuildInfo(t *testing.T) {
    br := commons.Branding{Flavor: "sherald", DisplayName:
        "System Herald"}
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.GET("/v1/healthz", HealthzHandler(br))
    req := httptest.NewRequest("GET", "/v1/healthz", nil)
    rec := httptest.NewRecorder()
    r.ServeHTTP(rec, req)
    if rec.Code != 200 {
        t.Fatalf("status = %d, want 200", rec.Code)
```

```
    }
    var body map[string]any
    if err := json.Unmarshal(rec.Body.Bytes(), &body); err !=
        nil {
        t.Fatalf("body unmarshal: %v", err)
    }
    if body["status"] != "ok" {
        t.Errorf("status field = %v, want \"ok\"",
        body["status"])
    }
    if build, ok := body["build"].(map[string]any); !ok ||
        build["version"] == "" {
        t.Errorf("build.version missing or empty — §107 bluff
        guard")
    }
}

func TestMetricsHandler_EmitsBuildInfoGauge(t *testing.T) {
    br := commons.Branding{Flavor: "sherald"}
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.GET("/metrics", MetricsHandler(br))
    req := httptest.NewRequest("GET", "/metrics", nil)
    rec := httptest.NewRecorder()
    r.ServeHTTP(rec, req)
    if rec.Code != 200 {
        t.Fatalf("status = %d, want 200", rec.Code)
    }
    body := rec.Body.String()
    if !strings.Contains(body, "sherald_build_info{") {

        t.Errorf("expected gauge sherald_build_info{...} in body,
        got:\n%s", body)
    }
}
```

Add imports: `"net/http/httptest"` + `"github.com/gin-gonic/gin"`.

- [ ] **Step 2: Run, verify FAIL**

```
go test ./commons/cli/ -count=1 2>&1 | tail -5
```

- [ ] **Step 3: Implement** `commons/cli/routes.go`

```
package cli

import (
```

```
    "fmt"
    "net/http"

    "github.com/gin-gonic/gin"

    "github.com/vasic-digital/herald/commons"
)

// Route declares one HTTP route. Wave 2 stubs return 501 with
        HRD pointer.
type Route struct {
    Method      string         // "GET", "POST", ...
    Path        string         // "/v1/compliance"
    Handler     gin.HandlerFunc
    Description string         // operator-readable
    HRD
        string         // "HRD-028" for 501 stubs; "" for live
        routes
}

// HealthzHandler returns 200 with {status:"ok",build:
        {version,go_version}}.
func HealthzHandler(br commons.Branding) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{
            "status": "ok",
            "flavor": br.Flavor,
            "build": gin.H{
                "version":    BuildVersion,
                "commit":     BuildCommit,
                "go_version": goVersionString(),
            },
        })
    }
}

// ReadyzHandler returns 200 with {status:"ready"}. Wave 2
        doesn't probe
// real readiness — flavor implementations may extend later.
func ReadyzHandler(br commons.Branding) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"status": "ready", "flavor":
        br.Flavor})
    }
}

// MetricsHandler returns Prometheus text with a
        <flavor>_build_info gauge.
```

```
// Wave 2 emits just the gauge; full Prometheus client wiring is
        HRD-NNN
// follow-up (live observability per spec §17).
func MetricsHandler(br commons.Branding) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Header("Content-Type", "text/plain; version=0.0.4;
        charset=utf-8")
        fmt.Fprintf(c.Writer, "# HELP %s_build_info Build
        information for %s.\n", br.Flavor, br.DisplayName)
        fmt.Fprintf(c.Writer, "# TYPE %s_build_info gauge\n",
        br.Flavor)
        fmt.Fprintf(c.Writer,
        "%s_build_info{version=%q,commit=%q,go_version=%q} 1\n",
            br.Flavor, BuildVersion, BuildCommit,
        goVersionString())
    }
}

// StubRouteHandler returns 501 with a JSON body citing the HRD
        that will
// implement the route. Used for Wave 2 placeholder routes.
func StubRouteHandler(route Route) gin.HandlerFunc {
    return func(c *gin.Context) {
        c.JSON(http.StatusNotImplemented, gin.H{
            "error":       "not yet implemented",
            "hrd":         route.HRD,
            "path":        route.Path,
            "description": route.Description,
        })
    }
}

func goVersionString() string {
    // Trivial wrapper so tests can mock; production uses
        runtime.Version().
    return "go" + buildGoVersion()
}
```

Add a tiny `commons/cli/build_info.go` (or inline in version.go) that exposes `buildGoVersion()` returning `runtime.Version()[2:]` (strips the leading "go").

- [ ] **Step 4: Update go.mod**

```
cd commons && go get github.com/gin-gonic/gin@latest && go mod
        tidy && cd ..
```

- [ ] **Step 5: Run tests, verify PASS**

```
go test ./commons/cli/ -count=1 -v 2>&1 | tail -20
```

- [ ] **Step 6: Commit**

```
git add commons/cli/routes.go commons/cli/cli_test.go commons/
    cli/build_info.go commons/go.mod commons/go.sum
git commit -m "Wave 2 step 3: commons/cli/ healthz + readyz +
    metrics handlers

Three Gin handlers + Route struct + StubRouteHandler.
- Healthz returns 200 with build info.
- Readyz returns 200 with {status:ready}.
- Metrics emits Prometheus text with <flavor>_build_info gauge.
- StubRouteHandler returns 501 with HRD pointer body for Wave 2
  flavor-specific stub routes.

§107: tests assert non-empty build.version + gauge name contains
flavor — empty values forbidden.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 4: `commons/cli/` — ServeCmd with Gin engine + graceful shutdown

Files:

- Create: `commons/cli/serve.go`
- Modify: `commons/cli/cli_test.go` (append integration-style test)

- [ ] **Step 1: Append failing test** (integration-style: actually starts the server in a goroutine, hits it, then cancels)

```
func TestServeCmd_BindsHealthzAndShutsDownOnSIGTERM(t
    *testing.T) {
    br := commons.Branding{Flavor: "sherald", DisplayName:
        "System Herald", DefaultPort: 0} // 0 = random free port
    opts := ServeOpts{Branding: br, Routes: []Route{}}
    cmd := ServeCmd(opts)
    // Bind to ephemeral port for test.
    cmd.SetArgs([]string{"--http-port", "0"})

    done := make(chan error, 1)
    ctx, cancel := context.WithTimeout(context.Background(),
        10*time.Second)
    defer cancel()
    cmd.SetContext(ctx)
```

```
    go func() { done <- cmd.Execute() }()
    // Polling for bind would be flaky without a callback — for
        unit test,
    // just verify the cmd doesn't return prematurely (still
        running after 200ms).
    select {
    case err := <-done:
        t.Fatalf("ServeCmd returned prematurely: %v", err)
    case <-time.After(200 * time.Millisecond):
        // good — server is running
    }
    cancel()
    select {
    case err := <-done:
        if err != nil && !errors.Is(err, context.Canceled) {
            t.Errorf("ServeCmd exit error = %v, want nil or
        context.Canceled", err)
        }
    case <-time.After(5 * time.Second):
        t.Fatal("ServeCmd did not shut down within 5s")
    }
}
```

Imports: `"context"`, `"errors"`, `"time"`.

- [ ] **Step 2: Run, verify FAIL**

- [ ] **Step 3: Implement** `commons/cli/serve.go`

```
package cli

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons"
)
```

```
// ServeOpts configures ServeCmd. Branding drives the default
        port +
// healthz/metrics gauge name. Routes are flavor-specific routes
        appended
// after the base healthz/readyz/metrics.
type ServeOpts struct {
    Branding commons.Branding
    Routes   []Route
}

// ServeCmd is the `<flavor>herald serve` subcommand for serving
        flavors.
// Binds a Gin engine with healthz/readyz/metrics + every Route
        in opts,
// listens on --http-port (default = branding.DefaultPort),
        graceful-
// shutdown on SIGTERM or context cancel. Exit 0 on clean
        shutdown.
func ServeCmd(opts ServeOpts) *cobra.Command {
    var port int
    cmd := &cobra.Command{
        Use:   "serve",
        Short: "Start the " + opts.Branding.DisplayName + " HTTP
        server",
        RunE: func(cmd *cobra.Command, args []string) error {
            if port == 0 {
                port = opts.Branding.DefaultPort
            }
            gin.SetMode(gin.ReleaseMode)
            r := gin.New()
            r.Use(gin.Recovery())
            r.GET("/v1/healthz", HealthzHandler(opts.Branding))
            r.GET("/v1/readyz", ReadyzHandler(opts.Branding))
            r.GET("/metrics", MetricsHandler(opts.Branding))
            for _, route := range opts.Routes {
                h := route.Handler
                if h == nil && route.HRD != "" {
                    h = StubRouteHandler(route)
                }
                r.Handle(route.Method, route.Path, h)
            }
            srv := &http.Server{
                Addr:    fmt.Sprintf(":%d", port),
                Handler: r,
            }
            errCh := make(chan error, 1)
            go func() {
```

```
                if err := srv.ListenAndServe(); err != nil &&
        err != http.ErrServerClosed {
                    errCh <- err
                }
            }()

            sigCh := make(chan os.Signal, 1)
            signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)
            defer signal.Stop(sigCh)

            ctx := cmd.Context()
            if ctx == nil {
                ctx = context.Background()
            }
            select {
            case err := <-errCh:
                return fmt.Errorf("listen: %w", err)
            case <-sigCh:
                // graceful shutdown
            case <-ctx.Done():
                // graceful shutdown
            }
            shutdownCtx, cancel :=
        context.WithTimeout(context.Background(), 5*time.Second)
            defer cancel()
            return srv.Shutdown(shutdownCtx)
        },
    }
    cmd.Flags().IntVar(&port, "http-port", 0, "TCP port to bind
        (default = flavor's DefaultPort)")
    return cmd
}
```

- [ ] **Step 4: Run tests, verify PASS**

```
go test ./commons/cli/ -count=1 -timeout=30s 2>&1 | tail -10
```

- [ ] **Step 5: Commit**

```
git add commons/cli/serve.go commons/cli/cli_test.go
git commit -m
        "Wave 2 step 4: commons/cli/ServeCmd — Gin engine +
        graceful shutdown

ServeCmd binds Gin engine with healthz/readyz/metrics +
        opts.Routes,
listens on --http-port (default = Branding.DefaultPort), SIGTERM
```

```
or context-cancel triggers graceful 5s shutdown. Returns nil on
clean shutdown, error on bind failure.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 5: Refactor pherald — version + stubs consume commons/cli/

Files:

- Modify: `pherald/cmd/pherald/main.go`, `version.go`, `stubs.go`, `migrate.go`, `wizard.go`
- Modify: `pherald/go.mod`

☐ **Step 1: Inspect existing pherald main.go**

```
cat pherald/cmd/pherald/main.go
```

Note the current branding-equivalent code + how subcommands are registered.

☐ **Step 2: Refactor `pherald/cmd/pherald/main.go` to use commons/cli/**

Replace the existing root construction with:

```
package main

import (
    "fmt"
    "os"

    "github.com/vasic-digital/herald/commons"
    "github.com/vasic-digital/herald/commons/cli"
)

func main() {
    br := commons.Branding{
        Flavor:      "pherald",
        Prefix:      "p",
        DisplayName: "Project Herald",
        DefaultPort: 24791,
        Mission:
        "Multi-mirror push + submodule propagation + project
        bindings",
    }
    root := cli.NewRootCmd(br)
    root.AddCommand(cli.VersionCmd(br))
```

```
    root.AddCommand(newServeCmd(br))            // pherald-
        specific (calls cli.ServeCmd internally)
    root.AddCommand(newMigrateCmd())            // migrate up/
        status/down/validate (unchanged)
    root.AddCommand(newWizardCmd())             // wizard
        credentials (unchanged)
    registerStubs(root)                          // §43 stub
        registrations

    if err := root.Execute(); err != nil {
        fmt.Fprintln(os.Stderr, "pherald:", err)
        os.Exit(1)
    }
}
```

☐

Step 3: Replace `pherald/cmd/pherald/stubs.go` **body**

```
package main

import (
    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons/cli"
)

// registerStubs adds every §43 command targeted at pherald as a
        501-stub.
// HRD-029 commit-push, HRD-030 submodule-propagate, HRD-043
        install-upstreams,
// HRD-044 fetch-guard, HRD-049 reopen, HRD-053 pre-push.
func registerStubs(root *cobra.Command) {
    root.AddCommand(cli.StubCmd("commit-push", "HRD-029",
        "Single-entrypoint locked commit + multi-mirror push
        (§2)"))
    root.AddCommand(cli.StubCmd("submodule-propagate",
        "HRD-030", "Owned-submodule walk in propagation order
        (§3)"))
    root.AddCommand(cli.StubCmd("install-upstreams", "HRD-043",
        "install_upstreams wrapper (§11.4.36)"))
    root.AddCommand(cli.StubCmd("fetch-guard", "HRD-044", "Pre-
        edit fetch + rebase enforcement (§11.4.37)"))
    root.AddCommand(cli.StubCmd("reopen", "HRD-049",
        "Issues→Fixed reversal + Reopens history (§11.4.55)"))
    root.AddCommand(cli.StubCmd("pre-push", "HRD-053", "Fetch +
        investigate + integrate hook (§11.4.71)"))
}
```

(Replace the old version+send+doctor+subscriber+deadletter stubs — those become subcommands of their own files if still needed.)

- [ ] **Step 4: Delete** `pherald/cmd/pherald/version.go`

The new `cli.VersionCmd(br)` replaces it. Delete the old file.

```
rm pherald/cmd/pherald/version.go
```

- [ ] **Step 5: Refactor** `pherald/cmd/pherald/migrate.go`

Update its `newMigrateCmd()` so it now coexists alongside `cli.VersionCmd(br)` in main.go. No body changes needed — migrate is its own thing; just verify it still compiles.

- [ ] **Step 6: Build + run pherald tests**

```
go build -o /tmp/pherald-refactor ./pherald/cmd/pherald
/tmp/pherald-refactor version --json | python3 -m json.tool
go test -race -count=1 ./pherald/... 2>&1 | tail -15
```

- [ ] **Step 7: Run e2e_bluff_hunt and verify E1-E13 still PASS (§107 regression check)**

```
bash scripts/e2e_bluff_hunt.sh 2>&1 | tail -30
```

Expected: 18/18 PASS (no regression).

- [ ] **Step 8: Commit**

```
git add pherald/cmd/pherald/main.go pherald/cmd/pherald/stubs.go
        pherald/go.mod
git rm pherald/cmd/pherald/version.go
git commit -m "Wave 2 step 5: refactor pherald to consume
        commons/cli/

main.go constructs a commons.Branding and uses cli.NewRootCmd +
cli.VersionCmd. stubs.go now registers §43 commit-push,
        submodule-
propagate, install-upstreams, fetch-guard, reopen, pre-push as
cli.StubCmd entries. Old version.go deleted (cli.VersionCmd
        replaces).

§107 regression: e2e_bluff_hunt E1-E13 still 13/13 PASS.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 6: Refactor pherald serve to consume cli.ServeCmd

Files:

- Modify: `pherald/cmd/pherald/serve.go` (or wherever the existing serve lives)
- Modify: `pherald/internal/http/routes.go`

☐ **Step 1: Inspect existing pherald serve**

```
find pherald -name '*.go' | xargs grep -l 'serve\|ServeCmd' 2>/
        dev/null | head
```

☐ **Step 2: Refactor serve.go to delegate to cli.ServeCmd with pherald routes**

```
package main

import (
    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons"
    "github.com/vasic-digital/herald/commons/cli"
)

func newServeCmd(br commons.Branding) *cobra.Command {
    return cli.ServeCmd(cli.ServeOpts{
        Branding: br,
        Routes: []cli.Route{
            {
                Method:      "POST",
                Path:        "/v1/events",
                HRD:         "HRD-016",
                Description: "Inbound CloudEvent ingestion (spec
        §41)",
            },
        },
    })
}
```

☐ **Step 3: Build + re-run e2e_bluff_hunt**

Expected: 18/18 still PASS — pherald still binds, healthz returns 200, /v1/events returns 501 with HRD-016 pointer.

```
bash scripts/e2e_bluff_hunt.sh 2>&1 | tail -20
```

☐ **Step 4: Commit**

```
git add pherald/cmd/pherald/serve.go pherald/internal/http/
git commit -m "Wave 2 step 6: pherald serve consumes cli.ServeCmd

pherald's serve subcommand delegates to commons/cli.ServeCmd with
        one
pherald-specific Route (/v1/events → HRD-016 501 stub). All Gin
plumbing, healthz/readyz/metrics, graceful-shutdown logic now
        lives in
commons/cli/. pherald is the first consumer of the shared
        scaffold.

E2E regression: 18/18 PASS (E1-E13 unchanged from pre-refactor).

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Tasks 7-12: Scaffold each of the 6 new flavors

Each task follows the SAME template. The implementer creates 4 files per serving flavor (3 for CLI-only) following this skeleton.

Task 7: sherald

Files:

- Create: `sherald/go.mod`
- Create: `sherald/cmd/sherald/main.go`
- Create: `sherald/internal/stubs/stubs.go`
- Create: `sherald/internal/http/routes.go`
- Modify: `go.work` (add `./sherald` to use list)

☐ **Step 1: Create** `sherald/go.mod`

```
module github.com/vasic-digital/herald/sherald

go 1.25.0

require (
    github.com/spf13/cobra v1.x.x
    github.com/vasic-digital/herald/commons v0.0.0
)

replace github.com/vasic-digital/herald/commons => ../commons
```

(`go mod tidy` after the source files exist will fill in the indirect deps.)

☐ **Step 2: Create** `sherald/cmd/sherald/main.go`

```
package main

import (
    "fmt"
    "os"

    "github.com/vasic-digital/herald/commons"
    "github.com/vasic-digital/herald/commons/cli"
    "github.com/vasic-digital/herald/sherald/internal/http"
    "github.com/vasic-digital/herald/sherald/internal/stubs"
)

func main() {
    br := commons.Branding{
        Flavor:      "sherald",
        Prefix:      "s",
        DisplayName: "System Herald",
        DefaultPort: 24793,
        Mission:     "Host safety + destructive-op intercept +
        mem-budget watcher",
    }
    root := cli.NewRootCmd(br)
    root.AddCommand(cli.VersionCmd(br))
    root.AddCommand(cli.ServeCmd(cli.ServeOpts{
        Branding: br,
        Routes:   http.Routes(),
    }))
    stubs.Register(root)
    if err := root.Execute(); err != nil {
        fmt.Fprintln(os.Stderr, "sherald:", err)
        os.Exit(1)
    }
}
```

- [ ] **Step 3: Create** `sherald/internal/stubs/stubs.go`

```
package stubs

import (
    "github.com/spf13/cobra"

    "github.com/vasic-digital/herald/commons/cli"
)

// Register adds all §43 command stubs targeted at sherald.
func Register(root *cobra.Command) {
```

```
    root.AddCommand(cli.StubCmd("destructive-guard", "HRD-033",
        "Wrap rm/git-reset/git-push-force with prerequisite
        checks (§9.1)"))
    root.AddCommand(cli.StubCmd("backup-snapshot", "HRD-034",
        "Hardlinked snapshot helper (§9.3)"))
    root.AddCommand(cli.StubCmd("constitution-pull", "HRD-040",
        "Wrap fetch + rebase + validation gate (§11.4.26)"))
    root.AddCommand(cli.StubCmd("force-push-gate", "HRD-046",
        "Merge-first + per-session-auth enforcement (§11.4.41)"))
    root.AddCommand(cli.StubCmd("mem-budget-watch", "HRD-056",
        "Daemon-mode 60% threshold watcher (§12.6)"))
}
```

- [ ] **Step 4: Create** `sherald/internal/http/routes.go`

```
package http

import "github.com/vasic-digital/herald/commons/cli"

// Routes returns sherald-specific HTTP routes. Wave 2 ships 1
        stub.
func Routes() []cli.Route {
    return []cli.Route{
        {
            Method:      "GET",
            Path:        "/v1/safety_state",
            HRD:         "HRD-098",
            Description: "Daemon status — open events, current
        mem%, last destructive-op log",
        },
    }
}
```

- [ ] **Step 5: Update** `go.work`

Append `./sherald` to the use block.

- [ ] **Step 6: Build, run, verify**

```
go build -o /tmp/sherald ./sherald/cmd/sherald
/tmp/sherald version --json | python3 -m json.tool
/tmp/sherald destructive-guard 2>&1 | head -3   # expect: error
        containing HRD-033
```

- [ ] **Step 7: Commit**

```
git add sherald/ go.work
git commit -m "Wave 2 step 7: sherald scaffold — System Herald
        flavor (serving)

5 §43 stubs (destructive-guard HRD-033, backup-snapshot HRD-034,
constitution-pull HRD-040, force-push-gate HRD-046, mem-budget-
        watch
HRD-056). Serves :24793 with healthz/readyz/metrics + /v1/
        safety_state
501 stub (→ HRD-098).

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 8: cherald

Same template as sherald. 11 §43 stubs (036/037/038/039/042/048/050/051/052/054/055). Serves :24792 with `/v1/compliance` 501 stub (→ HRD-028).

Task 9: iherald

Same template. 0 §43 stubs. Serves :24794 with `/v1/webhooks/page` 501 stub (→ HRD-024).

Task 10: bherald (CLI-only — no serve)

Same template MINUS `cli.ServeCmd(...)` registration MINUS `internal/http/`. 3 §43 stubs (035 evidence-capture, 041 test-tier-verify, 045 gate-retest).

Task 11: rherald (CLI-only)

Same as bherald. 3 §43 stubs (031 tag-mirror, 032 changelog-generate, 045 gate-retest).

Task 12: scherald (CLI-only)

Same as bherald. 1 §43 stub (047 status-digest).

For brevity I haven't expanded each task body — they are MECHANICAL repetitions of Task 7 with the flavor-specific Branding values + stub lists from the design doc's §2 table. Subagents executing these tasks MUST consult `docs/superpowers/specs/2026-05-21-wave2-flavor-scaffolds-design.md` §2 for the exact stub mapping.

Task 13: e2e_bluff_hunt E19-E33 + paired mutation tests

Files:

- Modify: `scripts/e2e_bluff_hunt.sh`
- Create: `tests/test_wave2_mutation_meta.sh`

- [ ] **Step 1: Add E19-E24 to e2e_bluff_hunt.sh**

After the E14-E16 block (or wherever appropriate), append:

```
echo ""
echo "== E19-E24: New flavor `version --json` (6 binaries) =="
for flavor in sherald cherald bherald rherald iherald scherald;
        do
    bin="/tmp/${flavor}-bluff-$$"
    if go build -o "${bin}" "./${flavor}/cmd/${flavor}" 2>/dev/
        null; then
        check "E1${flavor:0:1}? ${flavor} version --json returns
        valid JSON" \
            "\"${bin}\" version --json | python3 -c 'import
        sys,json; d=json.load(sys.stdin); assert
        d[\"flavor\"]==\"${flavor}\"; assert d[\"version\"];
        assert d[\"go_version\"]; print(\"ok\")'"
        rm -f "${bin}"
    else
        echo "FAIL  E1?: ${flavor} build failed"
        fail=$((fail+1))
    fi
done
```

(Adjust the numbering to E19-E24 explicitly per the design doc.)

- [ ] **Step 2: Add E25-E33 serve smokes**

For each of cherald + sherald + iherald: build the binary, start serve in background, curl healthz/readyz/flavor-specific stub, kill graceful, repeat.

- [ ] **Step 3: Update header comment "Eighteen invariants" → "Thirty-three invariants"**

- [ ] **Step 4: Create `tests/test_wave2_mutation_meta.sh`**

Three §1.1 mutation tests:

- M1: Strip the HRD pointer from sherald's StubCmd error message → E31 MUST FAIL.
- M2: Make VersionCmd return empty version → E19 MUST FAIL.
- M3: Make cherald serve on wrong port → E26 MUST FAIL.

Pattern from `tests/test_i8_usability_meta.sh` — apply mutation, run e2e probe, assert FAIL, restore.

- [ ] **Step 5: Run full bluff-hunt**

```
bash scripts/e2e_bluff_hunt.sh 2>&1 | tail -30
```

Expected: 33 PASS / 0 FAIL.

```
bash tests/test_wave2_mutation_meta.sh 2>&1 | tail -10
```

Expected: 3/3 mutation PASSes.

- [ ] **Step 6: Commit**

```
git add scripts/e2e_bluff_hunt.sh tests/
        test_wave2_mutation_meta.sh
git commit -m "Wave 2 step 13: e2e_bluff_hunt E19-E33 + paired
        mutations

15 new invariants: 6 flavor version probes (E19-E24), 3 serve
        binds
(E25-E27), 3 flavor-specific stub-route 501s (E28-E30), 1 stub-
        CLI
exit code + HRD-pointer probe (E31), 1 SIGTERM shutdown probe
        (E32),
1 pherald regression probe (E33).

Paired §1.1 mutations: strip HRD pointer from stub error → E31
        FAIL;
empty version field → E19 FAIL; wrong serve port → E26 FAIL. All
three mutation pairs PASS.

Total e2e_bluff_hunt: 18 → 33.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 14: Spec V3 r7 → r8 update + sibling regen

Files:

- Modify: `docs/specs/mvp/specification.V3.md` (5 sections per the design doc's Spec impact section)
- Regenerate: `docs/specs/mvp/specification.V3.{html,docx,pdf}`

- [ ] **Step 1: Bump revision in the spec's metadata table**

`| Revision | 8 |` (was 7). Update `Last modified` to today's date.

- [ ] **Step 2: Update §18 (Flavors)**

Add for each new flavor: Default port (where applicable), §43 stub count, serve surface notes.

- [ ] **Step 3: Update §41 (REST API surface)**

Add cherald's `/v1/compliance`, sherald's `/v1/safety_state`, iherald's `/v1/webhooks/page` to the route catalogue (with their HRD pointers).

- [ ] **Step 4: Add §44.M (Wave 2 milestone) subsection to §44 Foundation contract**

```
### §44.M Wave 2 — Flavor scaffolds (landed YYYY-MM-DD per
        [`docs/superpowers/specs/2026-05-21-wave2-flavor-
        scaffolds-design.md`])

[...summary of Wave 2 scope per the design doc...]
```

- [ ] **Step 5: Update §3.5 Branding**

Formalize the per-flavor Branding instance pattern.

- [ ] **Step 6: Regenerate siblings**

```
pandoc docs/specs/mvp/specification.V3.md -o docs/specs/mvp/
        specification.V3.html --standalone --toc --metadata
        title="Herald spec V3 r8"
pandoc docs/specs/mvp/specification.V3.md -o docs/specs/mvp/
        specification.V3.docx --toc --metadata
        title="Herald spec V3 r8"
pandoc docs/specs/mvp/specification.V3.md -o docs/specs/mvp/
        specification.V3.pdf --pdf-engine=weasyprint --toc --
        metadata title="Herald spec V3 r8"
```

- [ ] **Step 7: Commit**

```
git add docs/specs/mvp/specification.V3.{md,html,docx,pdf}
git commit -m "Wave 2 step 14: spec V3 r7 → r8 (Wave 2 captured)

§18 Flavors expanded with the 6 new flavor surfaces + ports +
        stub
counts. §41 REST surface adds cherald /v1/compliance, sherald
/v1/safety_state, iherald /v1/webhooks/page. §44 gets a new §44.M
Wave 2 milestone subsection. §3.5 formalizes the per-flavor
Branding instance pattern. §11.4.74 catalogue-checks gain HRD-092
commons/cli no-match entry.

All four siblings regenerated per §11.4.65.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

Task 15: Atomic Issues→Fixed + final anti-bluff + multi-mirror push

Files:

- Modify: `docs/Issues.md`, `docs/Fixed.md`, `docs/Status.md`
- Regenerate: siblings of each

- [ ] **Step 1: Open new HRDs (one per new flavor's scaffold + sherald /v1/safety_state)**

Add HRD-092..HRD-098 rows to `docs/Issues.md` Open table:

ID	Title
HRD-092	commons/cli/ shared scaffold (Wave 2 commons package)
HRD-093	sherald flavor scaffold
HRD-094	cherald flavor scaffold
HRD-095	bherald flavor scaffold
HRD-096	rherald flavor scaffold
HRD-097	iherald + scherald flavor scaffold (paired since iherald has no §43 stubs and scherald has 1)
HRD-098	sherald /v1/safety_state live (deferred per design doc §"Open questions")

- [ ] **Step 2: Mark HRD-092..HRD-097 as Fixed atomically (Wave 2 commits closed them)**

Move those rows from Issues.md Open to Fixed.md Recently-fixed in the same commit.

- [ ] **Step 3: Update Status.md r8 → r9**

Capture: Wave 2 complete, e2e_bluff_hunt 18 → 33, commons/cli/ landed, 6 flavor binaries operational, pherald refactor complete with E1-E13 unchanged.

- [ ] **Step 4: Regenerate all 3 siblings (Issues + Fixed + Status)**

```
for f in docs/Issues docs/Fixed docs/Status; do
    pandoc "${f}.md" -o "${f}.html" --standalone --toc
    pandoc "${f}.md" -o "${f}.docx" --toc
    pandoc "${f}.md" -o "${f}.pdf" --pdf-engine=weasyprint --toc
done
```

- [ ] **Step 5: Run FULL anti-bluff battery**

```
bash tests/test_constitution_inheritance.sh         # expect
        15/15
bash tests/test_constitution_inheritance_meta.sh    # expect
        META-PASS
```

```
bash tests/test_i6_refinement_meta.sh                # expect 3/3
bash tests/test_i8_usability_meta.sh                 # expect 5/5
bash tests/test_wave2_mutation_meta.sh               # expect 3/3
        (NEW)
bash scripts/audit_antibluff.sh                      # expect
        16/0/1 SKIP
bash scripts/codegraph_validate.sh                   # expect
        7/0/2 SKIP
bash scripts/e2e_bluff_hunt.sh                       # expect 33/0
```

ALL must be green.

☐ **Step 6: Re-index codegraph (per §11.4.79 — own-org submodules in scope)**

```
bash scripts/codegraph_setup.sh
bash scripts/codegraph_validate.sh
```

☐ **Step 7: Commit**

```
git add docs/Issues.{md,html,docx,pdf} docs/Fixed.
        {md,html,docx,pdf} docs/Status.
        {md,html,docx,pdf} .codegraph/
git commit -m
        "Wave 2 step 15: atomic Issues→Fixed (HRD-092..097) +
        Status r9

7 new HRDs opened (HRD-092 commons/cli, HRD-093..097 per-flavor
scaffolds, HRD-098 sherald /v1/safety_state live). HRD-092..097
moved atomically to Fixed.md (Wave 2 commits closed them in this
sequence). HRD-098 stays open per design doc §Open questions
(live implementation deferred to future iteration).

Status r8 → r9: Wave 2 complete; e2e_bluff_hunt 33/33; commons/
        cli
landed; 6 flavor binaries operational; pherald refactor with E1-
        E13
unchanged.

Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>"
```

☐ **Step 8: Multi-mirror push**

```
git push origin main
```

Expected: 4 lines (GitHub + GitLab + GitFlic + GitVerse) confirming.

Self-Review

1. Spec coverage check. Design doc requirements mapped:

Design req	Task
`commons/cli/` package	Tasks 1-4
pherald refactor	Tasks 5-6
6 new flavor binaries	Tasks 7-12
e2e_bluff_hunt E19-E33 + mutations	Task 13
Spec V3 r7 → r8	Task 14
Atomic Issues→Fixed + multi-mirror push	Task 15
Catalogue-Check `commons/cli/`	Task 14 §11.4.74 update

No design-section gaps.

2. Placeholder scan.

- "HRD-098" (sherald /v1/safety_state) — legitimate forward-declaration; Task 15 opens it explicitly.
- "TBD" — appears only in references to the upstream catalogue-check field; legitimate.
- `YYYY-MM-DD` in Task 14 commit message body — parametric placeholder for the engineer to fill at execution time. Legitimate.

3. Type consistency. `cli.Branding` → `commons.Branding` consistent across Tasks 1-12. `cli.Route`, `cli.ServeOpts` consistent.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-21-wave2-flavor-scaffolds.md`.

Per Universal Constitution §11.4.70 (subagent-driven default) + your "ALWAYS!!!" directive, the implementation proceeds via `superpowers:subagent-driven-development`. No execution-mode choice prompted — that's already settled by your constitutional mandate.

Next step: invoke `superpowers:subagent-driven-development` to dispatch task subagents.